GEMMA 4 DEVELOPER AGENT COMPETITION · PAPER TRACK · NOVEMBER 2026

Can a small local agent redo a cloud agent's work?

93 real tasks from a health-data repository that a cloud agent built, each verified by the competition's own evaluator, with an open harness and a Gemma 4 E4B baseline.

Health Transformation Lab · **Evidence you can check** SYNTHETIC PATIENT RECORDS

John Bowyer · personal project, not affiliated with any organization

WHO IT'S FOR, WHAT WE ASK, WHAT CHANGES

Built for the people who build and score coding agents

Who	What we ask them to do	What changes
Benchmark builders who mine tasks from a repository's history	Ask of every task: does its statement say what its tests check?	A score is quoted on tasks that can tell one agent from another
Agent builders working with small local models	Run their agent on the 93 tasks, with the competition's evaluator	A comparable score on health-data work, beside the popular libraries
Teams whose repositories are built with agents	Keep an issue, a pull request and a review for every change	Their own history can become a task set

The impact: a task set that says which of its tasks can measure an agent. So far it is one repository and one model, and the screen is an AI's, checked by a person in part.

A personal project: no partner has been contacted

THE PROBLEM

Coding benchmarks draw on a few popular libraries. Data work fails differently.

- SWE-bench draws on twelve popular Python libraries; the competition's development set, on four.
- In a **data repository** the costly defect is silent: a pipeline finishes and a number in its table is wrong.
- An **agent-built repository** can leave an unusually complete record: an issue, a pull request and a review for each change.

960
commits on `main`

775
of them marked as agent-authored

2,269
tests, run by CI on every pull request

240
pull requests, each with a committed review

One person in the loop with a cloud coding agent built this repository in fifteen days, without typing code. **What can a small local model take out of that loop?**

Source: the paper, Sections 1 and 3

WHAT WAS BUILT

A task set made by the competition's own rule, and checked by its evaluator



Controls. The evaluator resolves 93 of 93 reference patches and 0 of 93 empty patches.

An open harness. It gives the evaluator's verdict on every one of the model's patches compared: 25 of 25 and 23 of 23.

Each task carries a snapshot, a code graph, embeddings, and where its statement came from: an issue, a pull request or a commit message.

Source: the paper, Sections 4 and 5

WHAT THE SMALL MODEL DID

A few small fixes. More context did not reliably help.

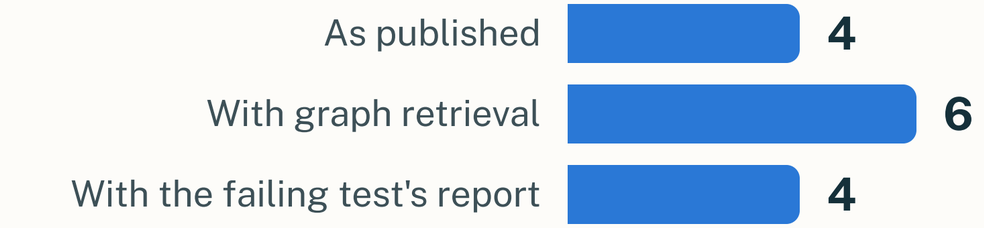
ON ITS OWN, FREE T4 GPUS

5 of 48

Gemma 4 E4B, cheapest tasks first (10.4%, 95% CI 4.5–22.2): a CI rule, a warning message, a label parser and two settings.

3 of 23 of the competition's own development tasks, under its evaluator.

PRE-REGISTERED EXPERIMENT · ALL 93 TASKS · ONE A100



No difference detected: exact McNemar p = 0.5 and 1.0. An unplanned second session on an L4 resolved 5 and 8.

The report comes from the hidden test: a test-first workflow's context, not a benchmark condition.

Source: the paper, Section 6

THE MAIN FINDING

The fixes were where the statement says what the tests check

8 of 17

core-split tasks were resolved in some of the five Colab runs (26 of 85 task-runs)

1 of 76

of the other tasks (1 of 380 task-runs)

The core split is the 17 tasks whose statements say what their tests check, by an AI screen of all 93. **Finding is not fixing:** where retrieval listed a function the fix edits (41 tasks), 5 of the core split's 7 were resolved in some run and none of the other 34.

DESCRIBED, NOT TESTED The second comparison was made after the runs. Source: the paper, Section 6

WHAT IT COST

Where a task's time went, and what a fix cost

TIME

66–83%

of a task's time was model calls, and 14–15% was context compaction (the Kaggle arms, from OpenTelemetry spans). 45 of 207 runs ended at the model's context window.

GPUS

1.9 and 1.8

GPU-hours a fix on four L4s, with graph retrieval and with the report: about 1.59 and 1.50 US dollars at a median rental of 0.83 an L4-hour. Kaggle's GPUs were free.

WHERE TO SPEND IT

0.38 vs 1.80

agent-hours a fix on the T4 baseline: the 13 core-split tasks it ran took 17% of its 9.0 agent-hours and gave 4 of its 5 fixes.

What it means: the model's own calls are where the time goes, not the graph tools, and the core-split tasks are where an hour of this model bought the most.

Source: the paper, Section 6, "Time, GPUs and cost". The prices are rental estimates, not money spent

WHAT TO DO WITH IT

Use the set. Quote results on the core split.

Run your agent on it. 93 tasks, CC BY 4.0, in the competition's layout: its evaluator runs on them as published.

Quote results on the core split. Over every run, 9 of the 10 distinct tasks this model has resolved are among its 17.

Screen your own tasks. Ask of each one whether its statement says what its tests check, before a score is quoted on it.

Check any number. Each one in the paper has a row in `paper/sources.md` naming the file it came from.

Tasks: kaggle.com/datasets/jcblearning/health-transformation-lab-coding-agent-tasks
Paper, sources and agent configuration: github.com/healthtransformationlab/gemma-developer-agent

Drill-down

Backup slides for questions. The full detail is in the paper.

- What this does not show
- This set beside the competition's own tasks
- How each number is kept honest

DRILL-DOWN · LIMITS

What this does not show

One new repository, compared with the competition's development set.

No cloud baseline. The cloud agent that made these changes, with a person in the loop, was not rerun on the statements alone.

A small experiment. At a baseline of 4 of 93 only a large effect could show.

No leaderboard score. Every run under the competition's evaluator used 5 minutes and 40 tool calls, not 60 and 100, once per arm and machine.

AI screens. A person checked 42 verdicts, the core split's 17 among them, in summary and after seeing the AI's; 51 have AI ratings only.

Prompted repair. Each task comes with a failing test: noticing a silent defect unprompted is not measured.

Read the results as: what one small model did on one repository's tasks, with the screen's verdicts checked by one person in part.

Source: the paper, Section 7

DRILL-DOWN · THE COMPETITION'S OWN TASKS

The two sets differ in their tests more than their statements

	This set	Development set
The fix edits a function the graph holds	51 of 93	114 of 129
Of those: a name or call edge reaches it; with embedding search too	22; 42	52; 66
Statement vague or worse (AI screen)	19 of 93	11 of 40
Tests need a name or text the statement never gives	**76 of 93**	**14 of 40**
Resolved as published, on Kaggle	0 of 21	3 of 23

On the competition's tasks the graph holds more of the fixes and the screen passes half. The screen and the agent took a seeded 40 of its 129 development tasks; a Kaggle notebook can verify 23 of them.

Source: the paper, Section 6, "Beside the competition's own tasks"

DRILL-DOWN · EVIDENCE YOU CAN CHECK

How each number is kept honest

A source for every number. `paper/sources.md` has a row for each figure in the paper, naming the file it came from, and keeps the rows of text that was cut.

A plan before the results. The experiment's plan was hashed before any result, and each addendum before its run started (source repository, issues 674 and 687).

Checked on every change. In CI, `paper/check_rules.py` checks the paper against the competition's rules, and `docs/build_decks.py` checks this deck's numbers against the paper.

What went wrong is in the paper. Colab's network was open to the agent: 3 of the five runs' 687 commands ran an installer, in tasks no run resolved, and none left its workspace.

Not a result: anything from a dry run or a sample is labelled as one, and the unplanned second session is described beside the registered one, never added to it.

Sources: paper/sources.md, paper/rules_check.md and the paper, Section 6